Chapter 1: Introduction

1. What four components does the IEEE include in its definition of software, and why is each important?
2. Compare and contrast Generic software with Custom software in terms of development cost, time to market, and fit to user needs.
3. How does Student Software differ from Professional (industrial-strength) Software regarding reliability, maintenance, and expected lifespan?
4. Identify and explain the "Four P's" (People, Process, Product, Project) that influence software engineering success.
5. What factors should an organization weigh in the Build-or-Buy decision, and what are the main pros and cons of each option?

Chapter 2: Software Methodologies

1. Define "software methodology" and list its key elements (e.g. roles, activities, work products).
2. Outline the main phases of the software process life cycle (Definition, Analysis, Design, Implementation, Testing, Support) and state the goal of each.
3. Compare the Waterfall model with the Incremental model in terms of adaptability to changing requirements and delivery cadence.
4. Choose two values from the Agile Manifesto and explain how each contrasts with traditional, document-driven approaches.
5. Describe three core Extreme Programming (XP) practices (e.g. Test-Driven Development, Pair Programming, Continuous Integration) and discuss their benefits.

Chapter 3: Customer Requirements

1. What is the goal of the requirements gathering (Specification) phase, and why is clear requirements capture crucial to project success?
2. Explain the standard User Story template

   > As a [role] I want [goal] so I can [reason]
   > and describe the purpose of each part.

3. Compare and contrast the two estimation techniques: estimate by analogy versus Planning Poker. When might you choose one over the other?
4. List the "Definition of Ready" criteria that a User Story should satisfy before a team commits to implementation.

5. Describe the Given–When–Then format in Behavior-Driven Development (BDD) and explain how it helps bridge the gap between requirements and automated acceptance tests.

Chapter 4: Project Plan

1. What are the three phases of the XP Planning Game, and what happens in each phase?
2. Describe the five levels of planning and explain how each level connects to the others.
3. How does the Product Roadmap differ from the Release Plan in terms of content and purpose?
4. What role does velocity play in iteration planning in XP, and how is it calculated?
5. What are the responsibilities of the Business and the Development team during an Iteration Planning meeting?

Chapter 5: Software Modeling

1. What is the purpose of a UML Use Case Diagram, and how are Actors and Use Cases represented?
2. Compare Use Cases and User Stories in terms of their level of detail and main objectives.
3. What are CRC cards (Class–Responsibility–Collaborator), and how are they used to discover and define classes?
4. In a UML Class Diagram, what is the difference between Association, Aggregation, and Composition?
5. How do Sequence Diagrams complement Class Diagrams in modeling system behavior over time?

Chapter 6: Development

1. Describe the steps of the XP developer's daily routine—Pair-up, Quick Design, Write Test, Code, Refactor, Integrate, Go Home—and explain the importance of each step.
2. What are the benefits and drawbacks of Pair Programming according to the University of Utah study?
3. Explain the concept of Collective Ownership and how it impacts code quality and team dynamics.
4. Why are Coding Standards necessary in an XP environment, and what key characteristics should they include?
5. Define Refactoring and list three code smells that signal its need.

Chapter 7: Testing

1. What is the difference between Unit Tests and Acceptance Tests in XP, and what role does each play in ensuring software quality?
2. Explain the Test-Driven Development (TDD) cycle and its benefits for improving reliability.
3. How are Test Fixtures (@Before, @After, @BeforeClass, @AfterClass) used in JUnit to manage setup and teardown?
4. What are the main JUnit assert methods, and how do assertions for floating-point values differ?
5. Compare White-Box Testing and Black-Box Testing in terms of test design and objectives.